Estrutura de Dados

Curso Tecnologia em Análise de Desenvolvimento de Sistemas

Aula 09

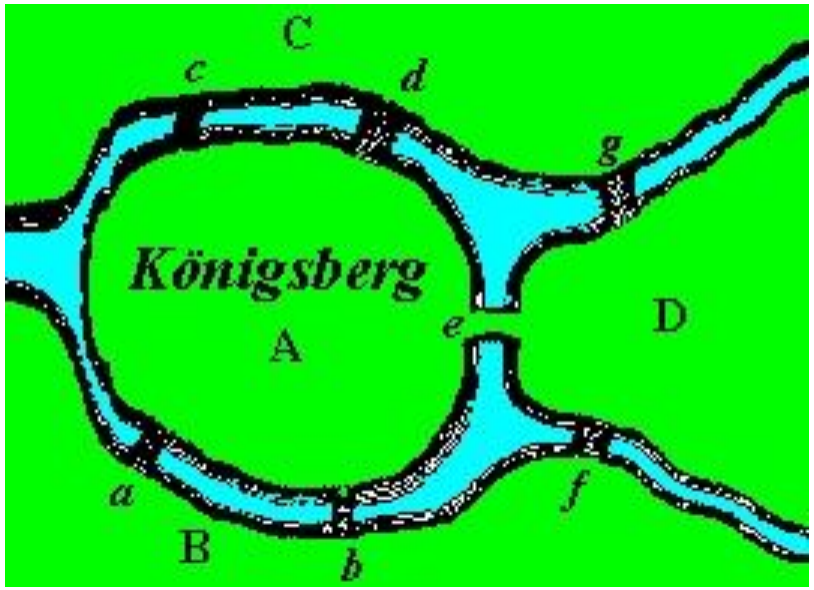
Prof. Claudio Benossi

1.
Unidade

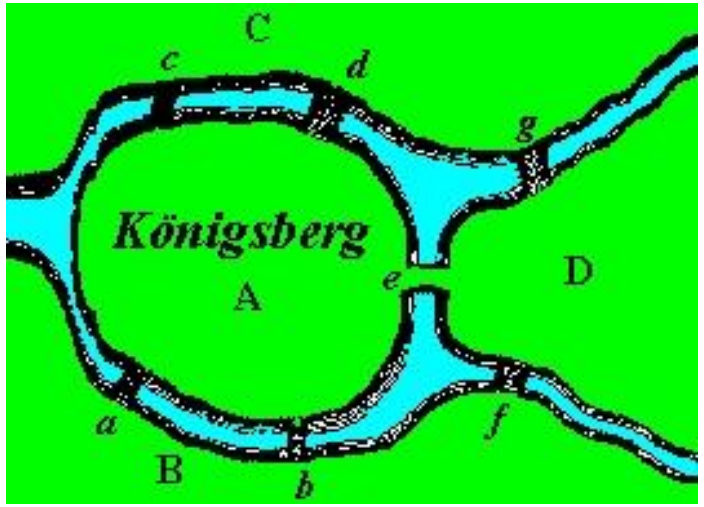
Grafos

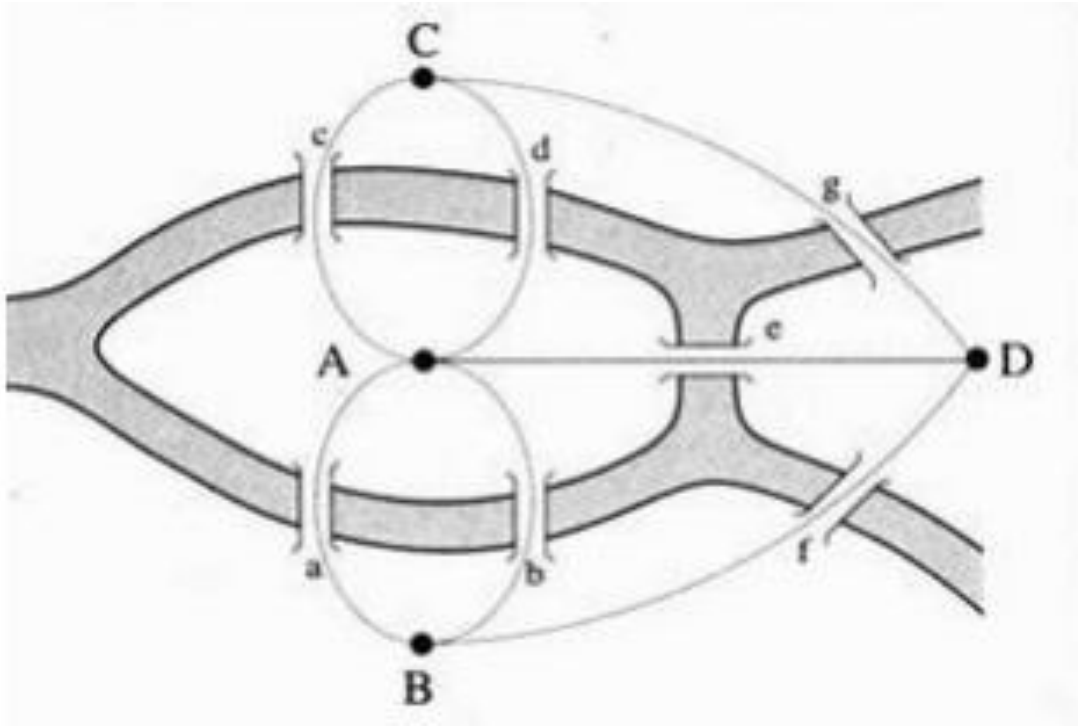
Grafos

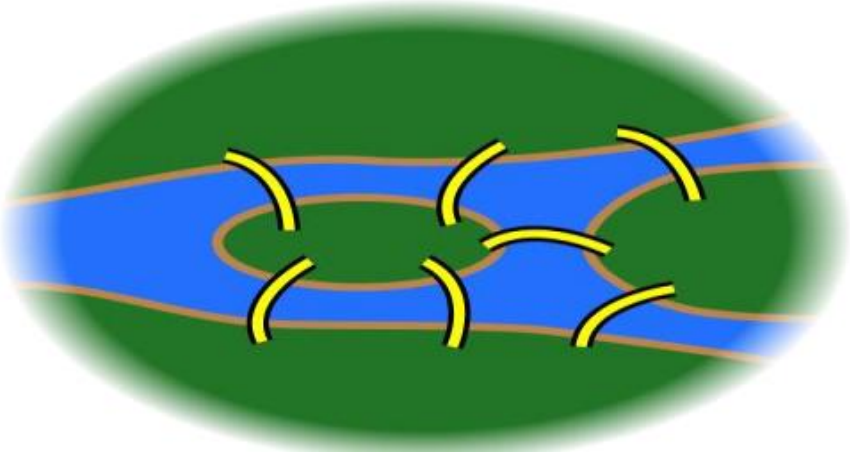
A origem da teoria dos Grafos, de acordo com alguns autores, **Königsberg** foi a maior cidade oriental da Alemanha até a Segunda Guerra Mundial e tinha sete pontes (marcadas de a a g no esboço), e os habitantes da cidade se perguntavam se era possível passear pela cidade de maneira a atravessar cada uma das sete pontes exatamente uma vez. O que você acha?



Grafos



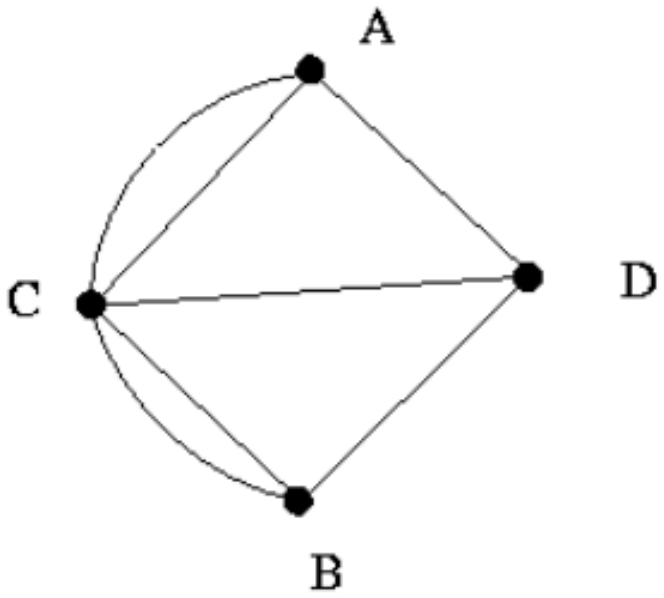




Grafos

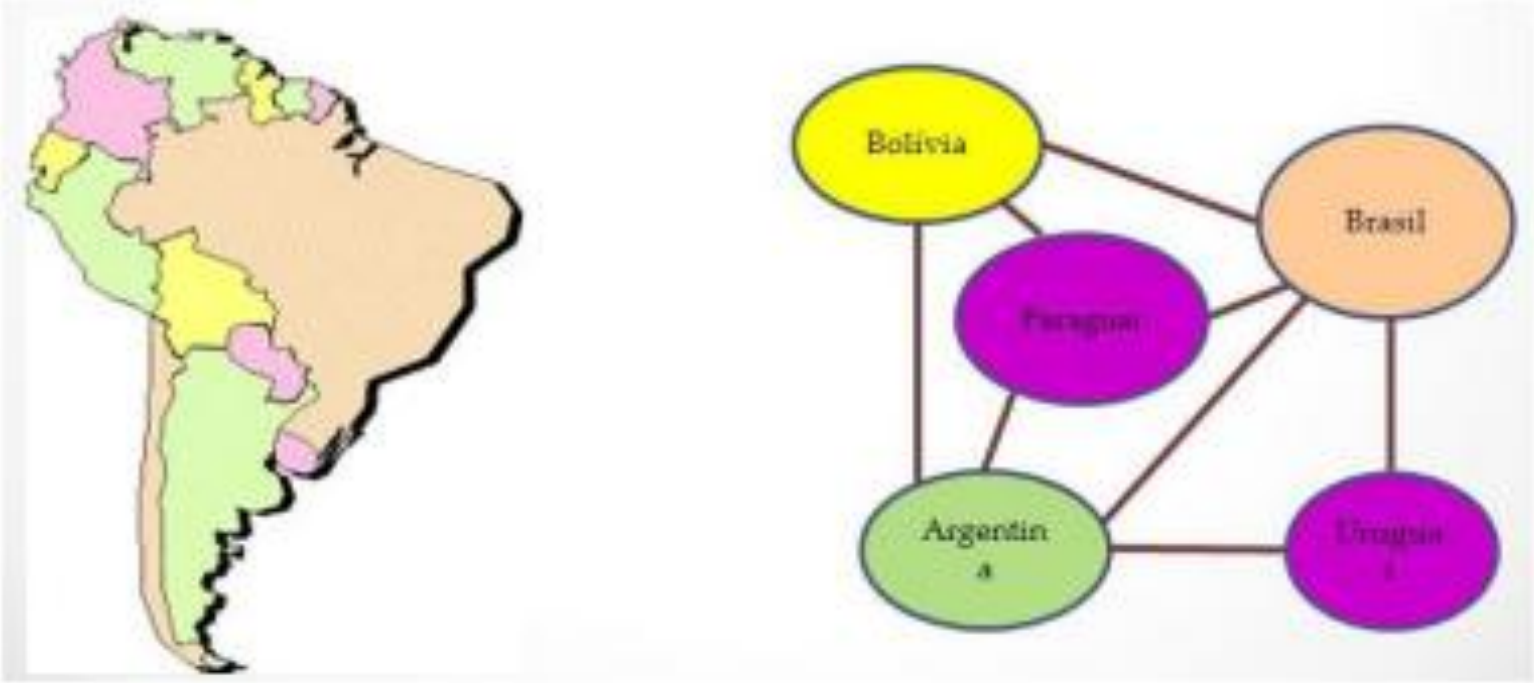
Para resolver esse problema, Euler o representou com o grafo ilustrado à direita. Com essa representação, e considerando as propriedades do grafos que serão apresentadas, é possível resolver facilmente esse problema.

matemático suiço
Leonhard Euler (1707-1783)



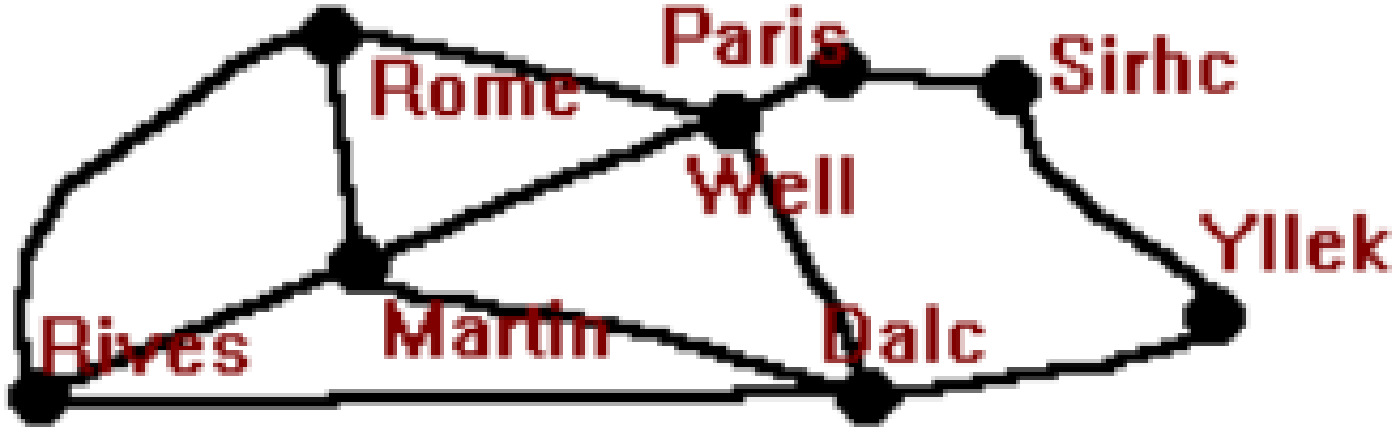
Grafos

Outro problema: Quantas cores precisamos para colorir um mapa de modo que dois países vizinhos sejam coloridos com cores diferentes?



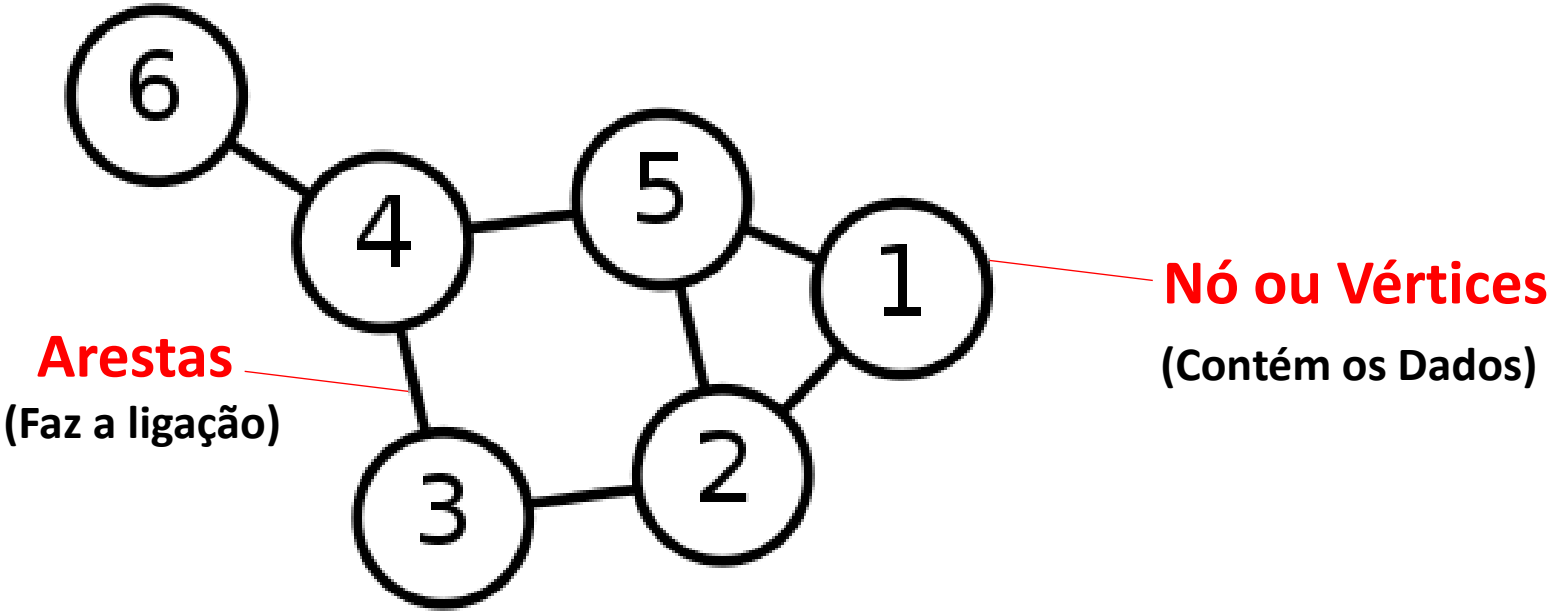
Grafos

Dado um mapa de várias cidades e as estradas entre elas, é possível que um vendedor itinerante visite (passe por) cada uma das cidades exatamente uma vez?



Grafos

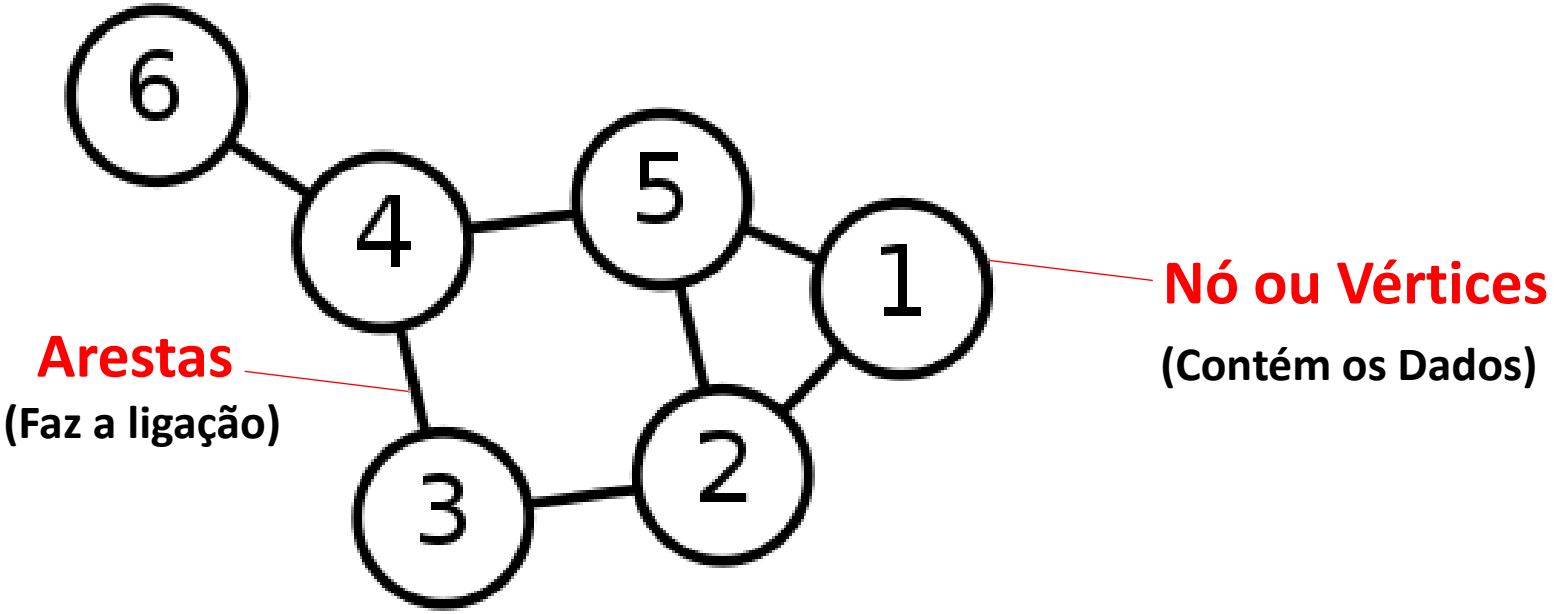
Definição: Um gráfo G é feito de vértices e arestas e consiste em um conjunto V de nós e um conjunto E de arestas, cada qual "une" dois dos nós.



Grafos

Definição: Um gráfo é uma estrutura de dados que visa trabalhar com relações entre elementos.

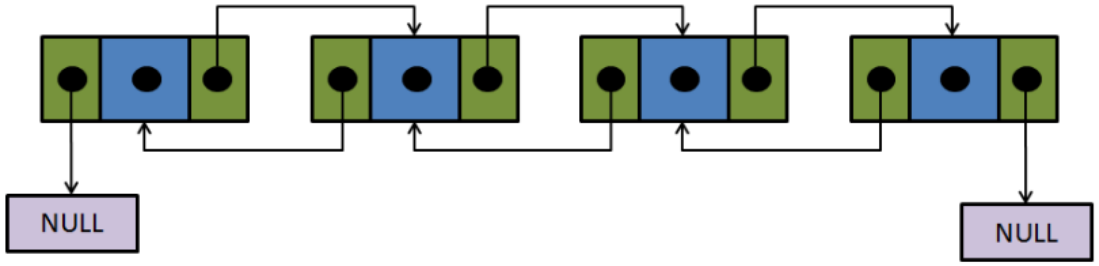
Exemplo clássico a internet.

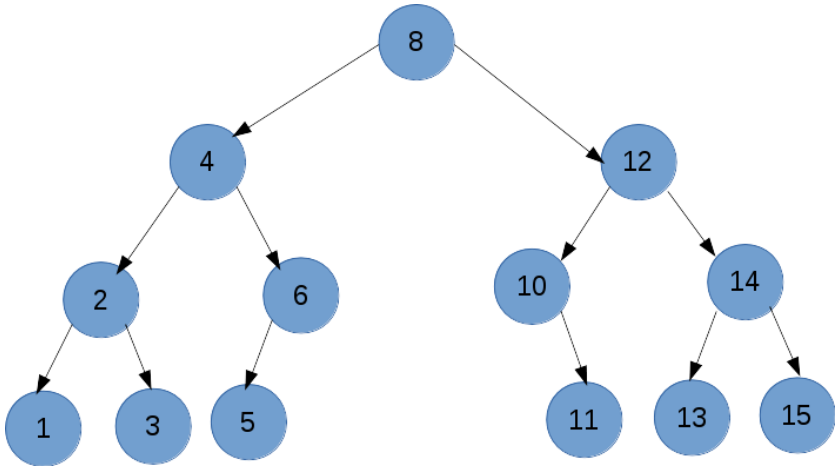


Grafos

Então podemos dizer que uma Lista Encadeada, uma Árvore Binaria também são considerados também como grafos.

Tanto a Lista Encadeada com a Árvore binária tem regras, como por exemplo uma Árvores Binaria um vértice só pode estar ligado a somente outros dois vértices.

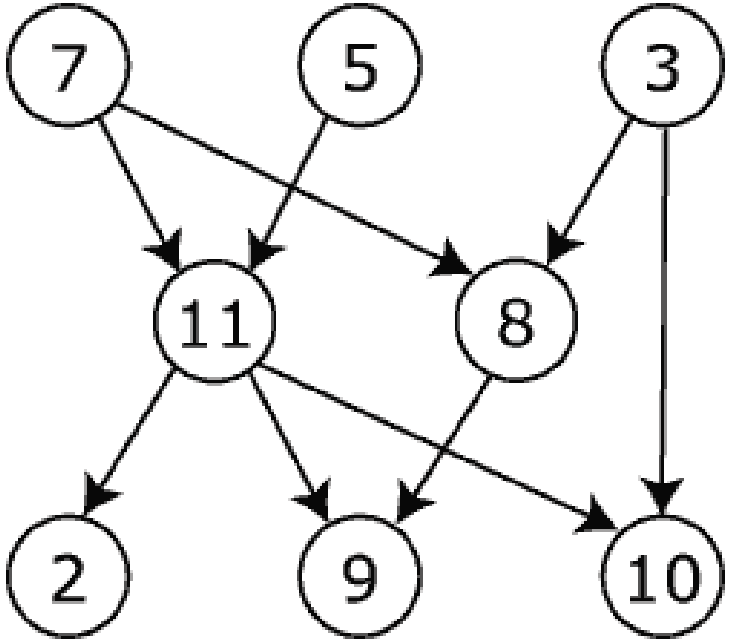


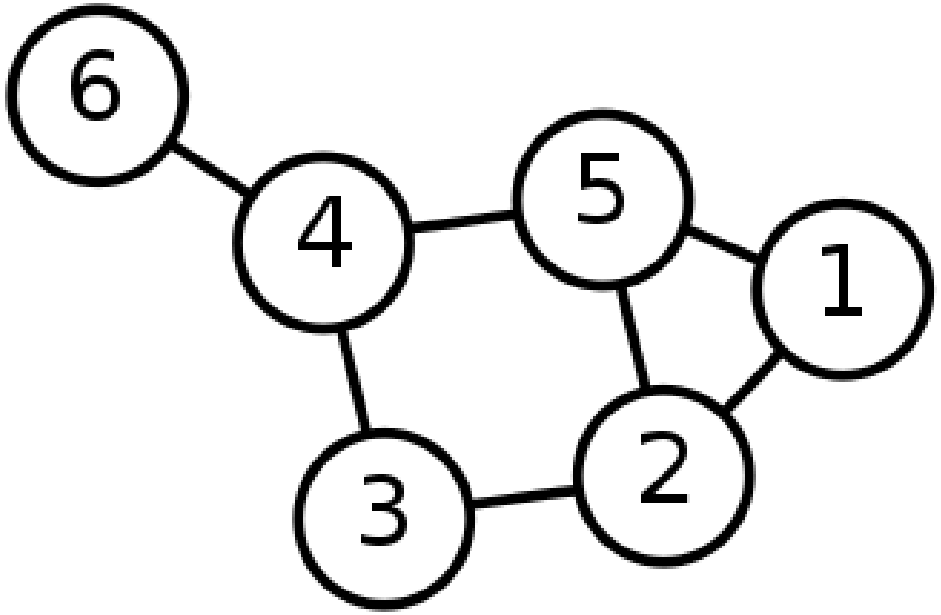


Grafos

É usado para representar estruturas complexas.

Pode existir Grafo Direcionado e Grafo Não Direcionado:

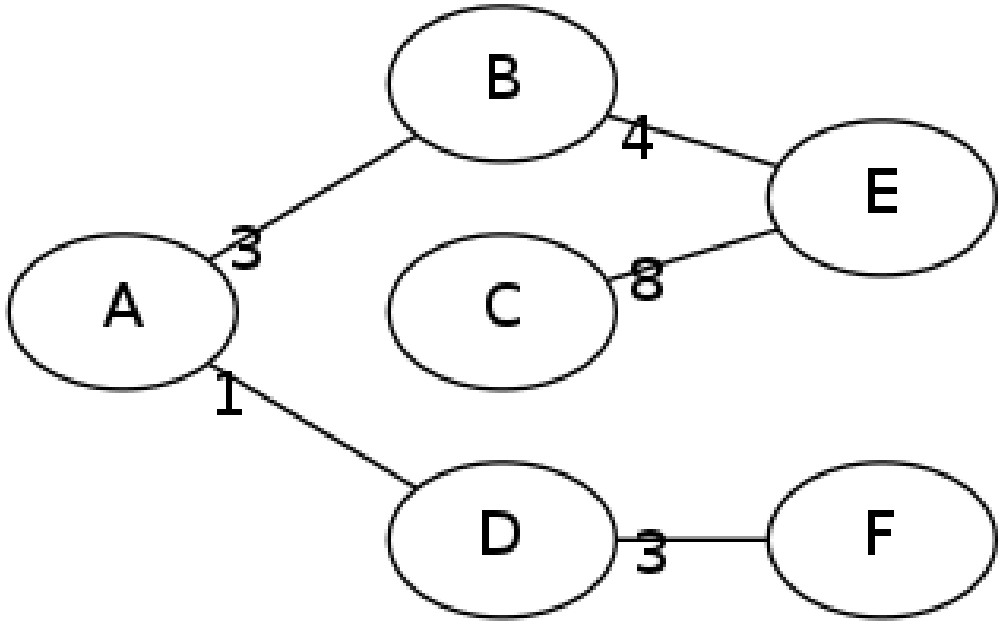


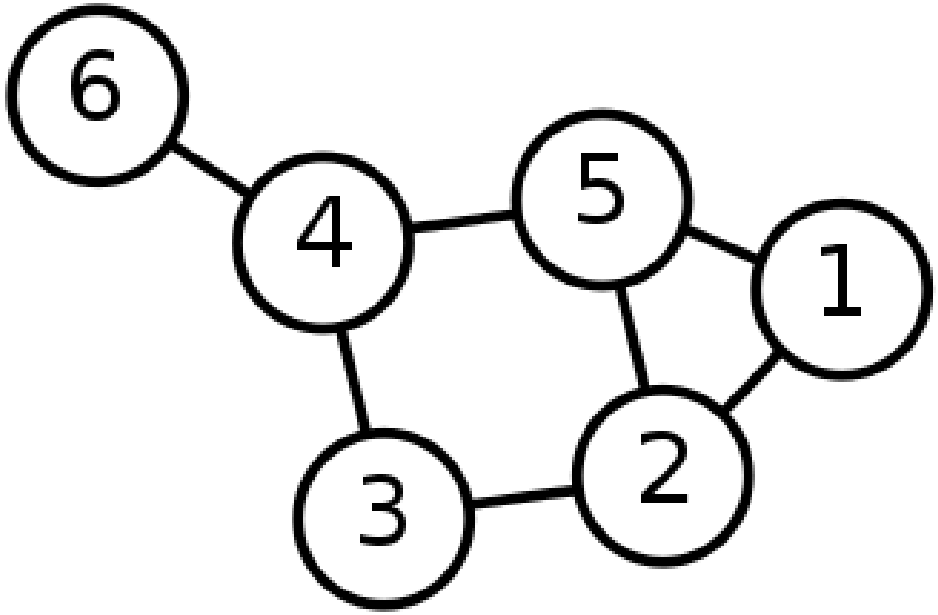


Grafos

Ainda podemos determinar o peso para aresta.

Pode existir Grafo com peso e Grafo sem peso:



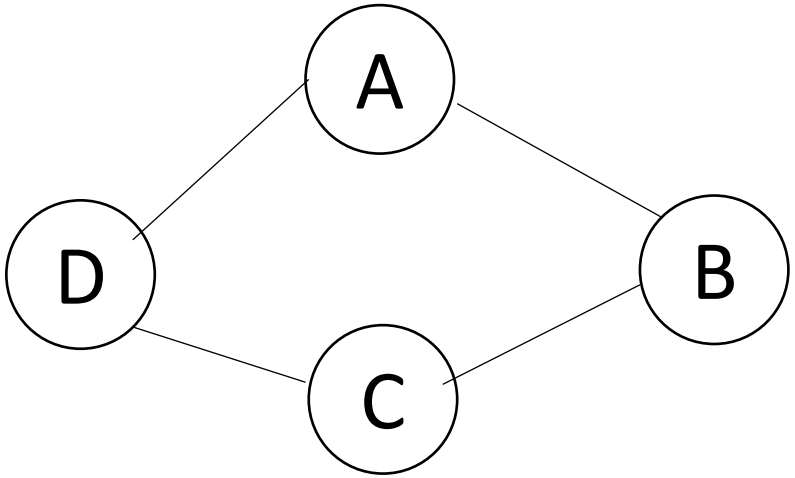


Grafos

Ainda podemos determinar o peso para aresta.

Grau: número de associações incidentes em um vértice;

Dígrafos diferenciam grau de saída de grau de entrada, os somando se obtêm o grau total de um vértice.



Todos os elementos tem grau = 2

Grafos

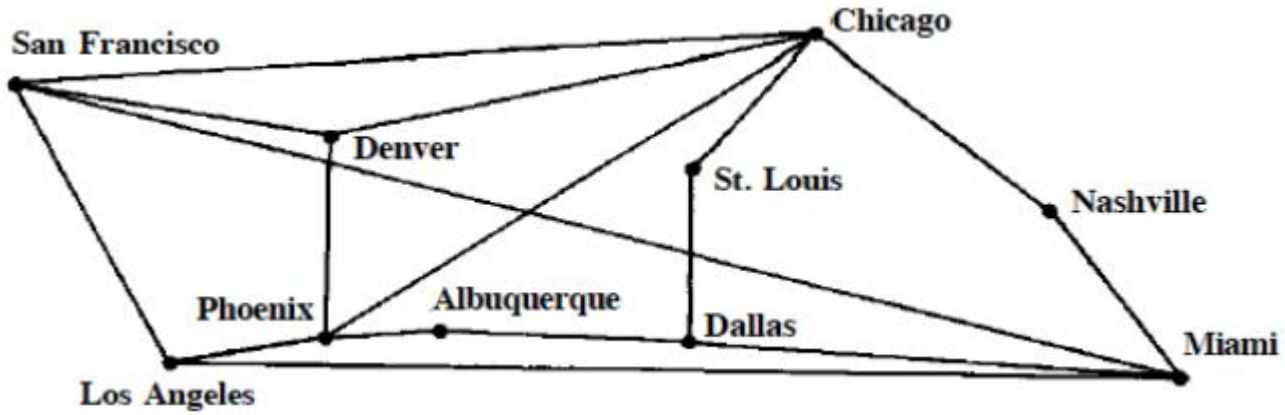
É usado para representar estruturas complexas.

Assim, representamos uma aresta $e \in E$ como um subconjunto de dois elementos de V: $e = \{u, v\}$ para alguns $u, v \in V$, onde chamamos u e v os fins da aresta e.

Grafos

Grafo é um conjunto de pares ordenados (V,E) onde:

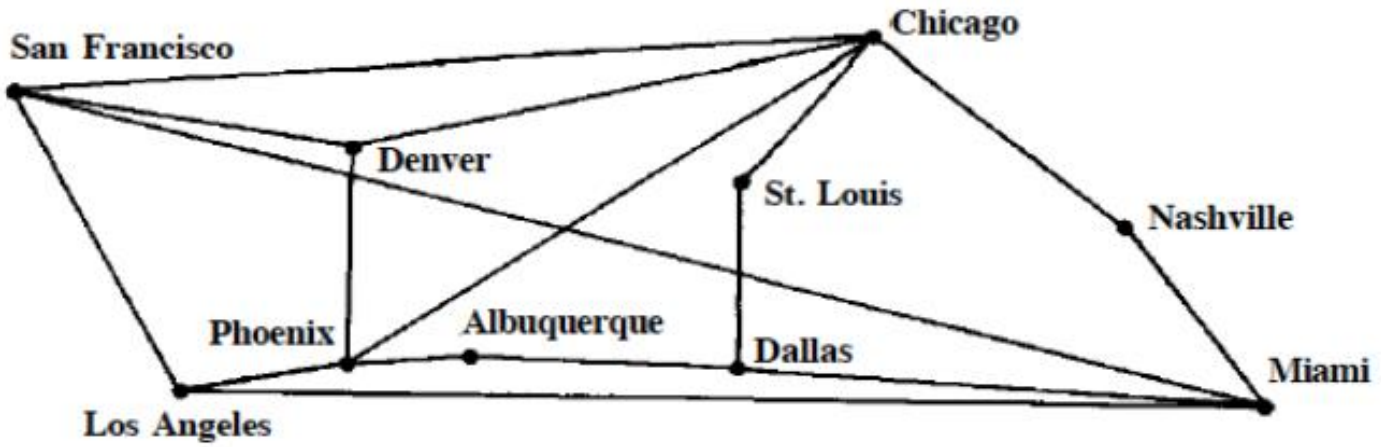
- V é um conjunto não-vazio e finito de vértices (nós ou nodos); e
- E é um conjunto finito de arestas ou arcos (em inglês, edge) que conecta os vértices do grafo,



Grafos

Grafo é um conjunto de pares ordenados (V,E) onde:

V = {Albuquerque, Chicago, Dallas, Denver, Los Angeles, Nashville, Miami, Phoenix, San Francisco, St. Louis}



Rotulando as arestas do Grafo

V = {v1, v2, v3, v4, v5, v6}

E = {a1, a2, a3, a4, a5, a6, a7, a8}

onde

a1 = (v1,v2)

a2 = (v1,v3)

a3 = (v1,v3)

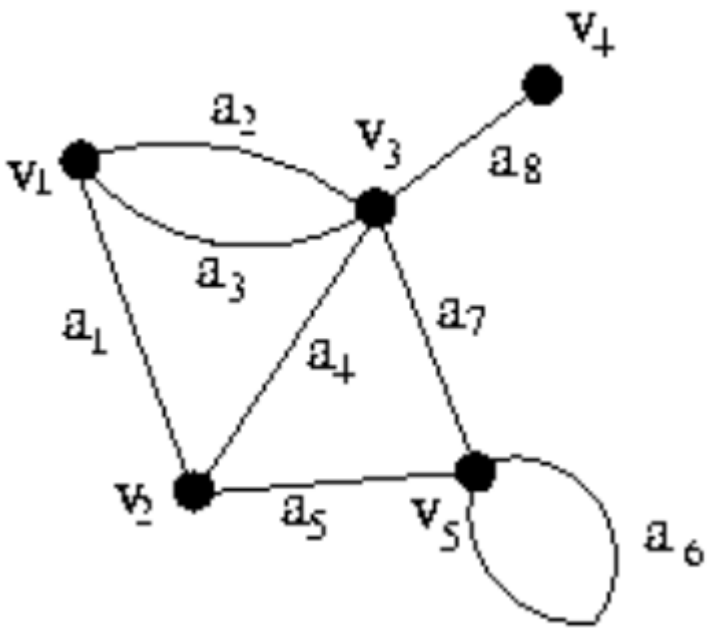
a4 = (v2,v3)

a5 = (v2,v5)

a6 = (v5,v5)

a7 = (v3,v5)

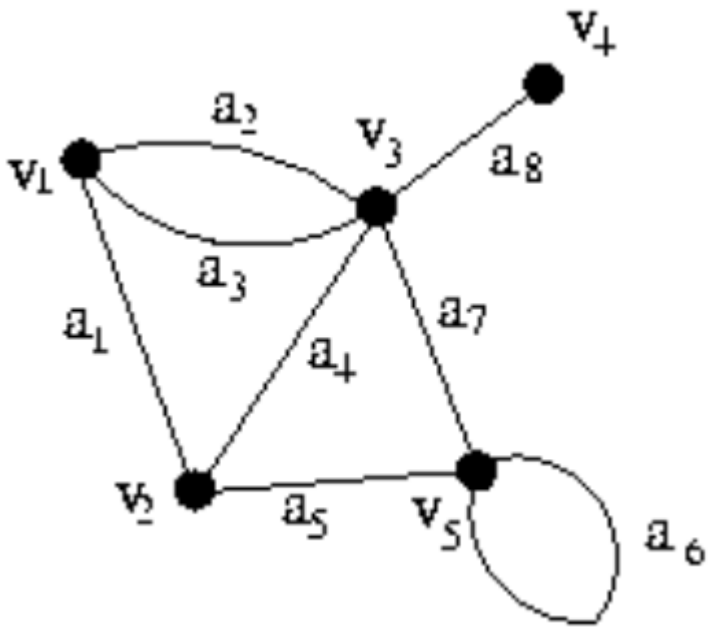
a8 = (v3,v4)



Rotulando as arestas do Grafo

V = {v1, v2, v3, v4, v5, v6}

E = { (v1,v2), (v1,v3), (v1,v3), (v2,v3), (v2,v5), (v5,v5), (v3,v5), (v3,v4)}



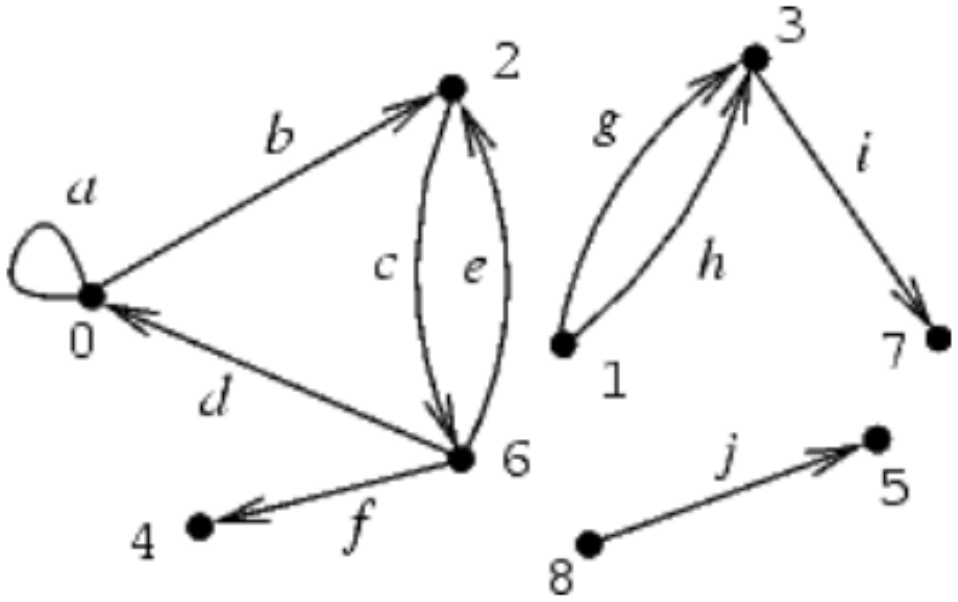
Rotulando as arestas do Grafo

V: {0, 1, 2, 3, 4, 5, 6, 7, 8}

E={a, b, c, d, e, f, g, h, i, j},

onde:

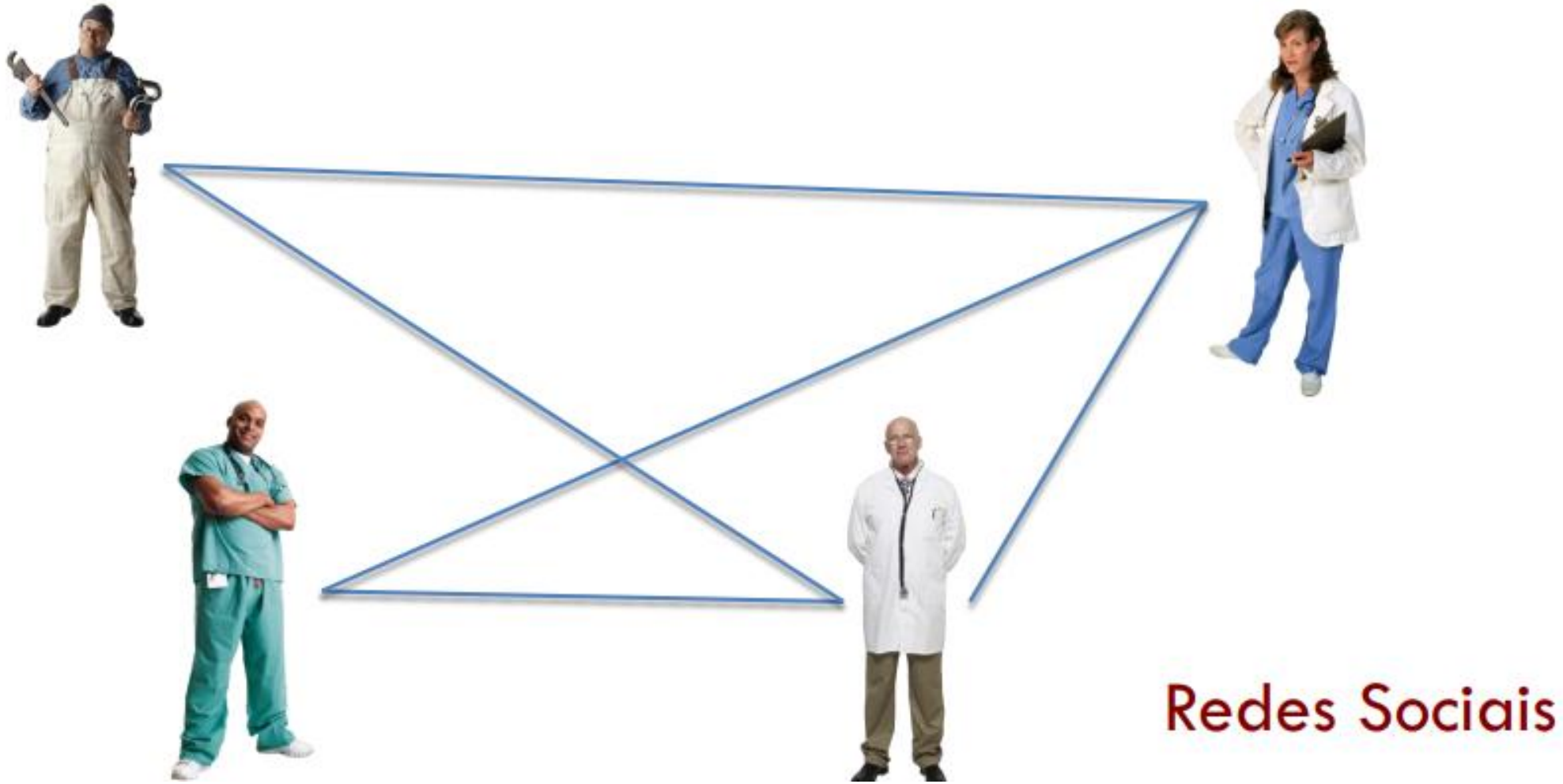
a = (0,0)	f = (6,4)
b = (0,2)	g = (1,3)
c = (2,6)	h = (1,3)
d = (6,0)	i = (3,7)
e = (6,2)	j = (8,5)



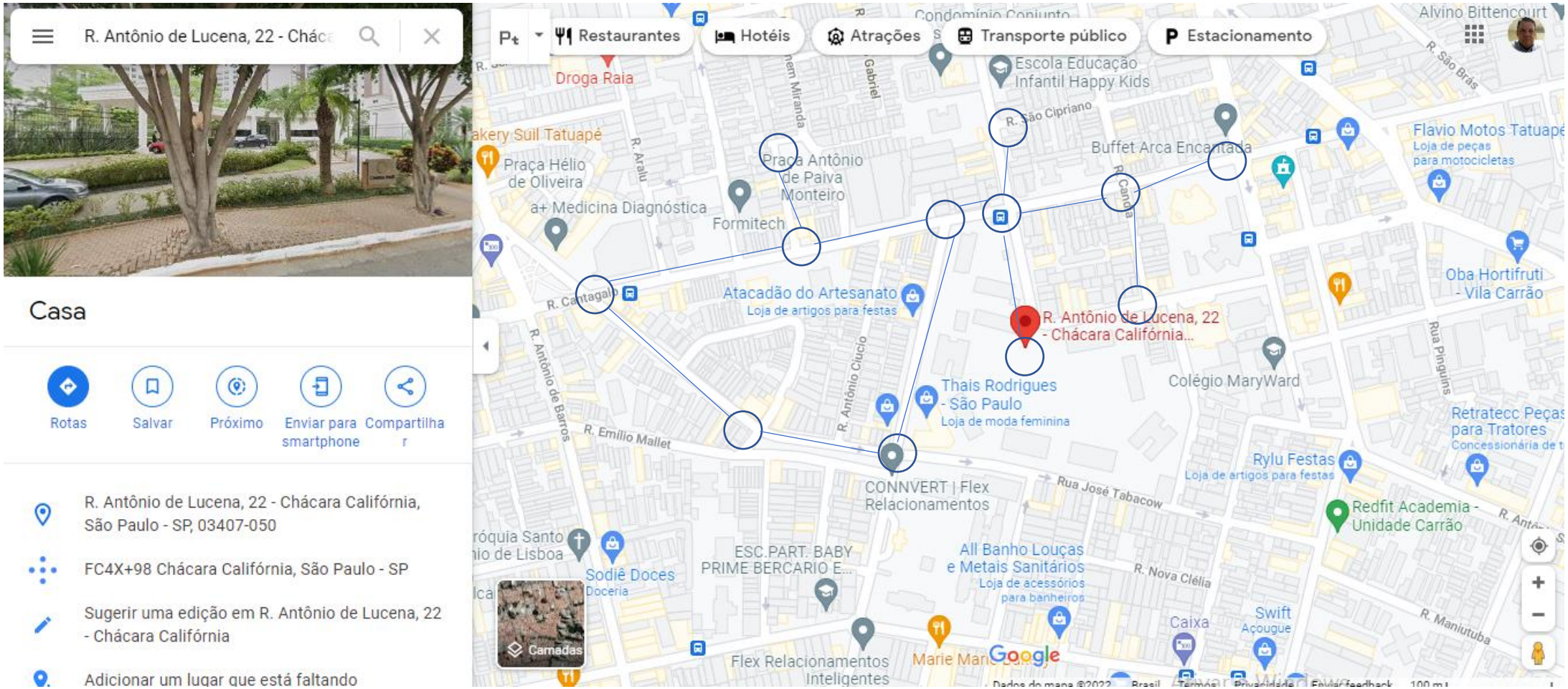
Aplicações em Grafos

- Sistemas de Georeferenciamento;
- Redes Sociais;
- Modelagem de circuitos digitais;
- Representação de processos em sistemas paralelos, distribuídos ou Redes de Computadores;
- Redes Semânticas;
- Recuperação da Informação;
- Dentre outras.

Abstrações de relacionamentos entre os objetos



Grafo para representação de Mapas de Ruas

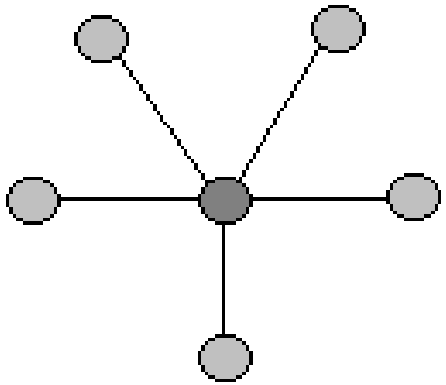


Posso usar peso de arestas, direção, etc

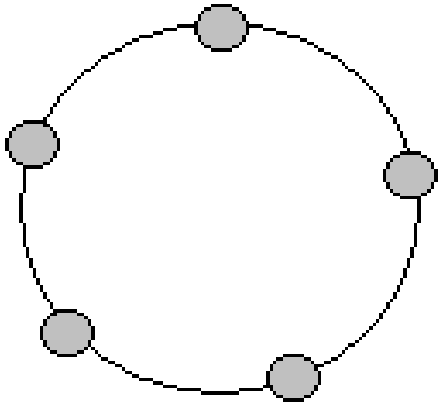
Cada esquina você pode pensar como um Vértice e as Ruas como Arestas que faz a ligação

Grafo para representação de Rede Social

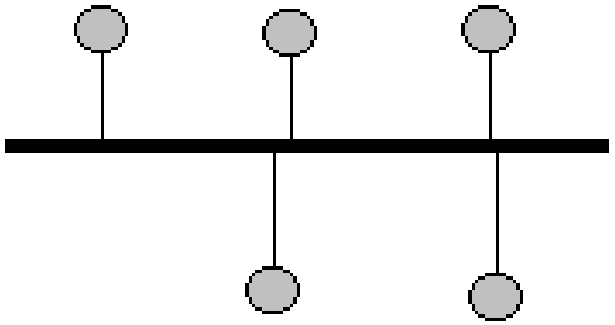
Grafos para representação das Topologias de Redes de Computadores



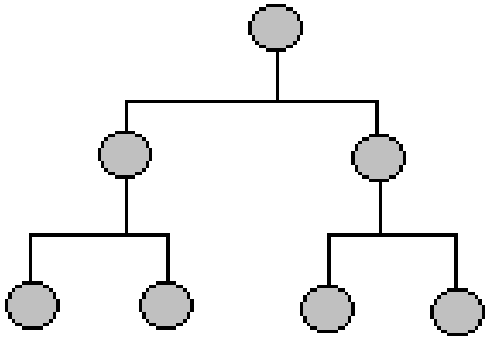
Estrela



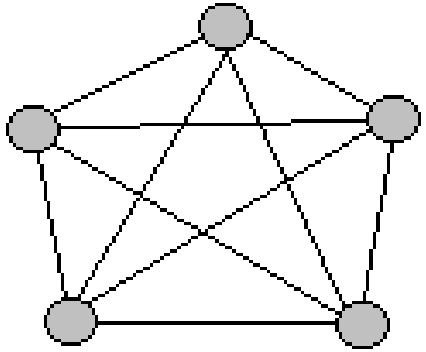
Anel



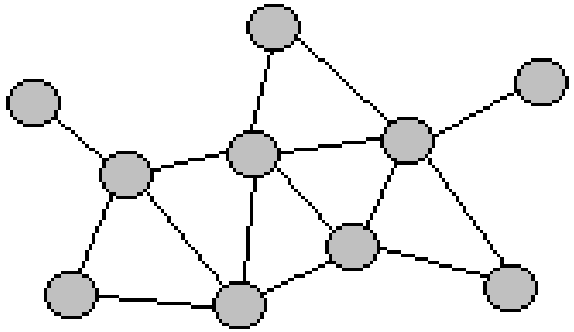
Barramento



Árvore

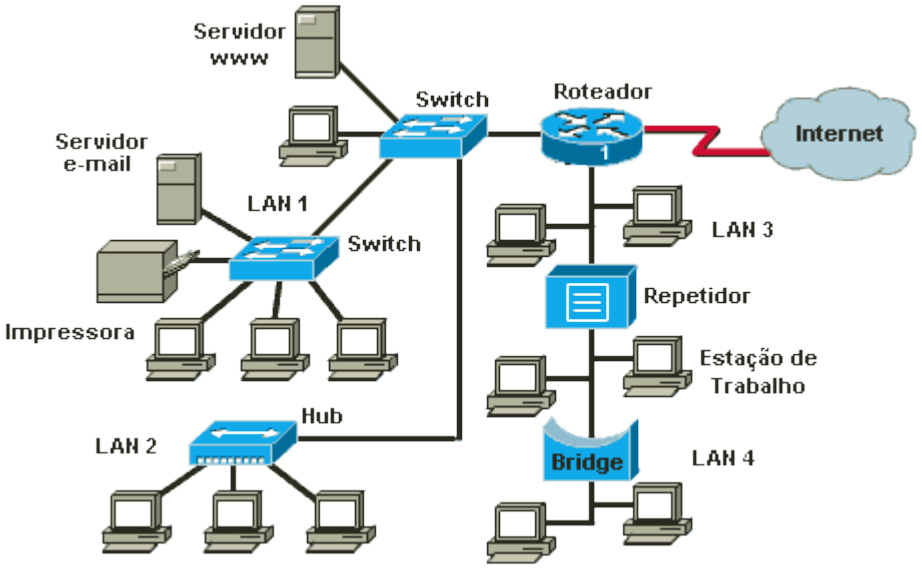


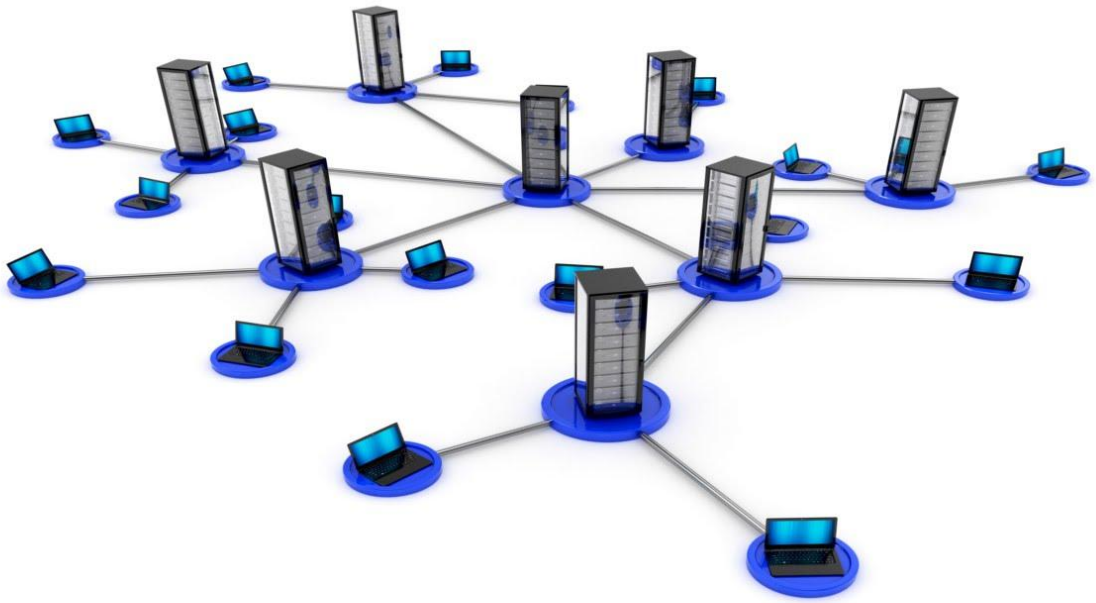
Ponto a Ponto



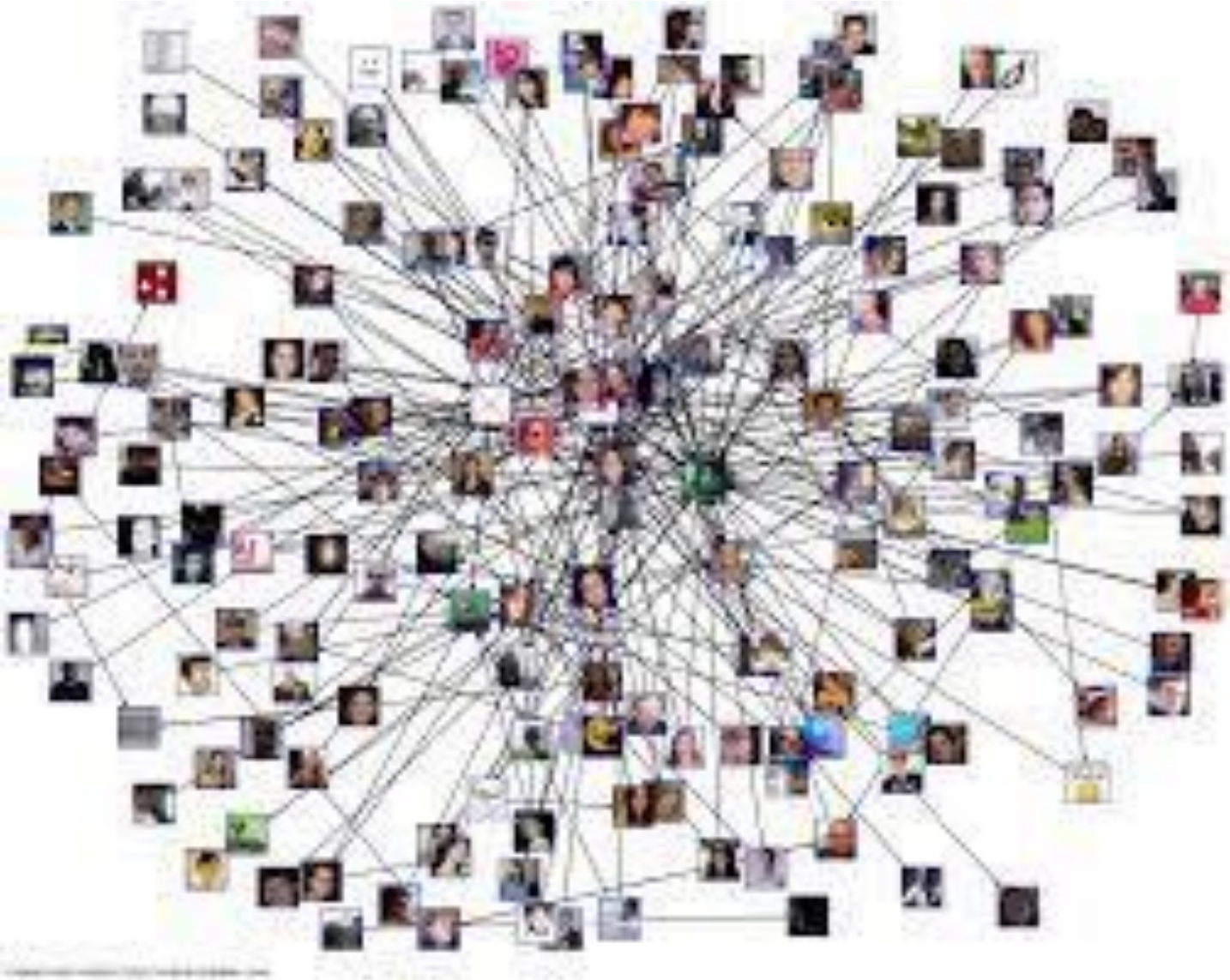
Mesh

Grafos para representação de Redes de Computadores





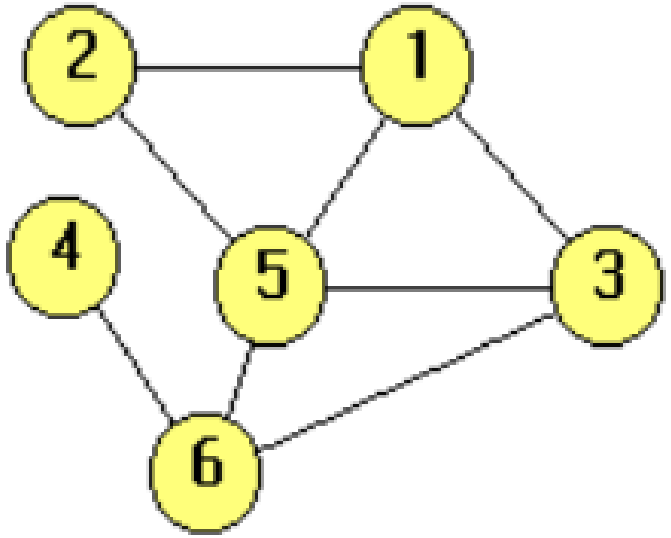
Redes Complexas



Grafo Não-Direcionado e Dígrafo

Grafos não-direcionados (ou sem orientação)

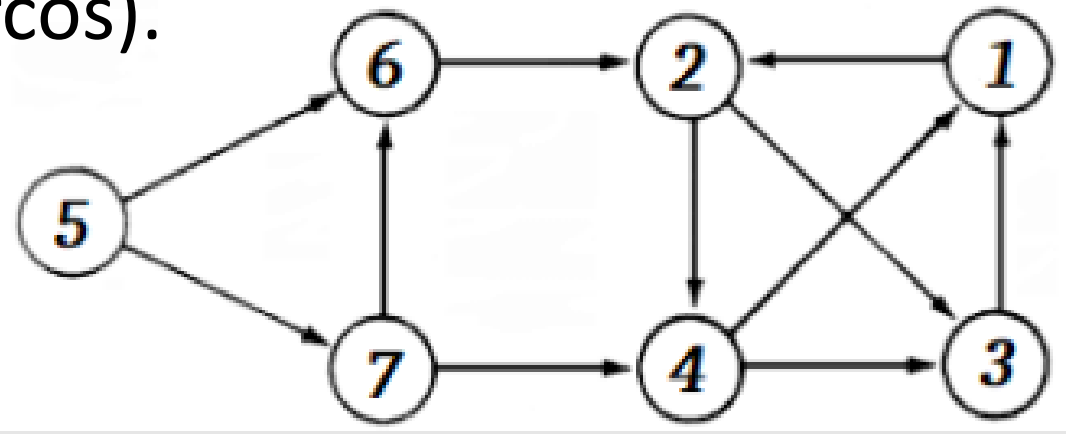
- As arestas (u,v) e (v,u) são consideradas uma única arestas.
- Self-loops não são permitidos.
- É um tipo especial de grafo simétrico.



Grafo Não-Direcionado e Dígrafo

Grafo direcionado ou Dígrafos (com orientação)

- Uma aresta parte de um vértice origem e alcança um vértice destino.
- Podem existir self-loops.
- As arestas possuem orientação (arcos).

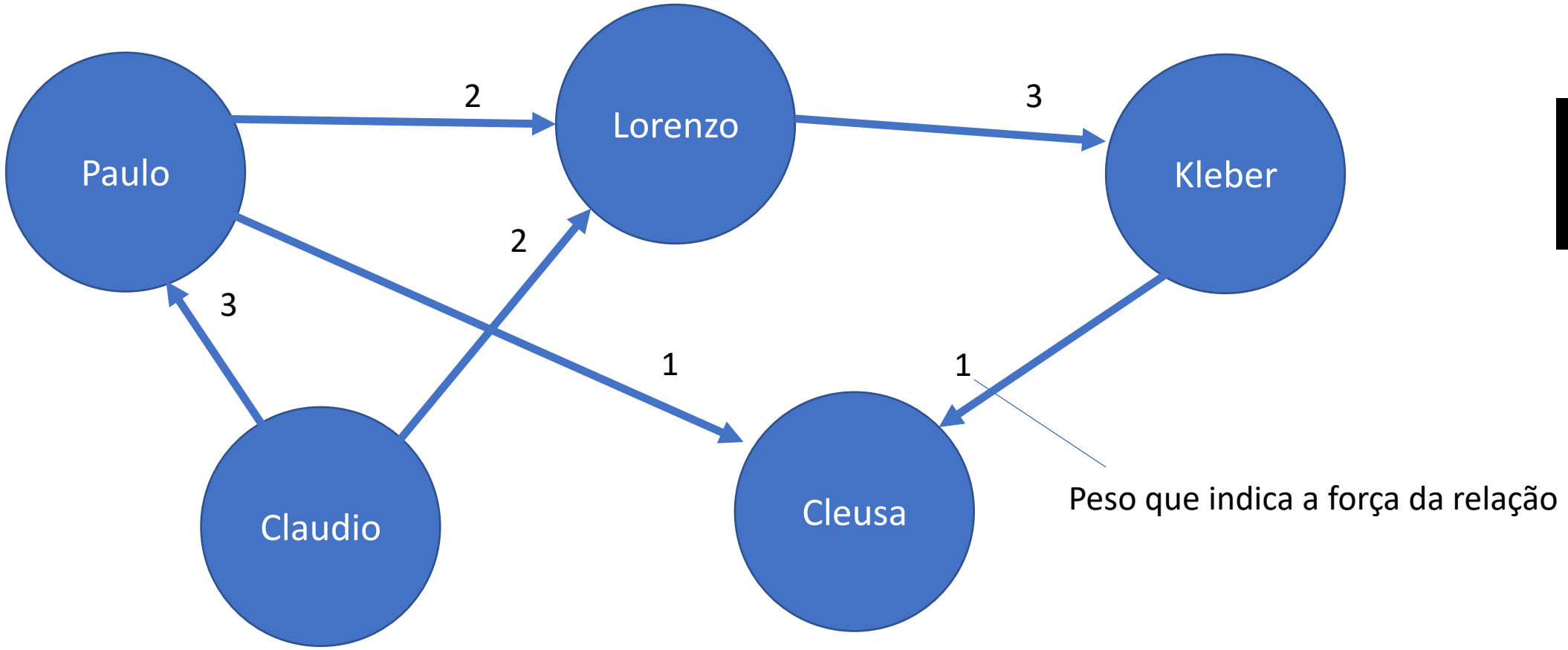


Terminologias

- Dois **vértices** em um grafo são ditos adjacentes se forem os extremos de uma mesma aresta.
- Um **laço** em um grafo é uma aresta com extremos n-n para algum nó n.
- Duas arestas que tenham os mesmos extremos são chamadas de **arestas paralelas**.
- Um **grafo simples** é um grafo que não tem arestas paralelas e nem laços.
- Um **vértice isolado** não é adjacente a qualquer outro vértice

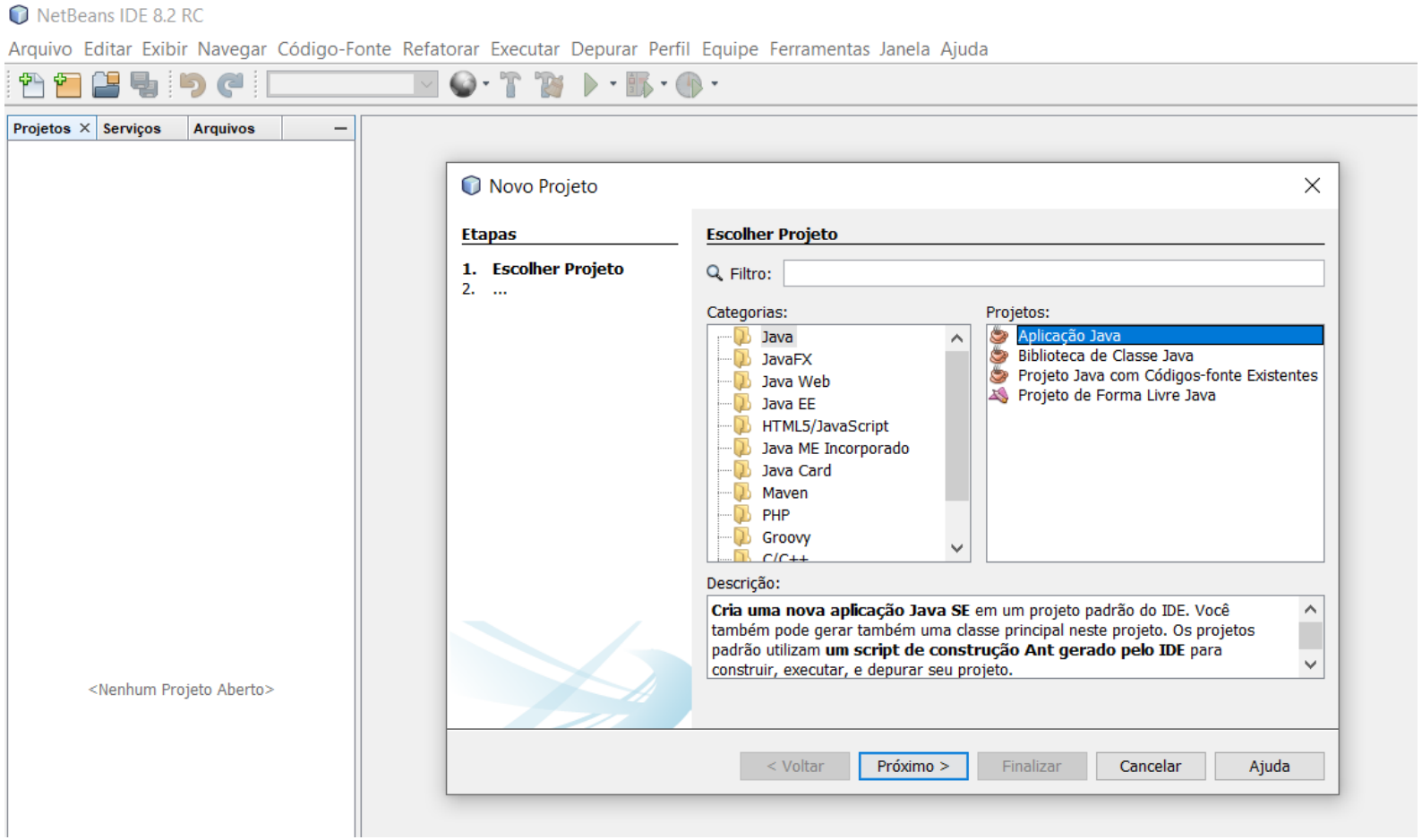
Grafos

Vamos implementar na Linguagem Java o seguinte grafo de amizades:



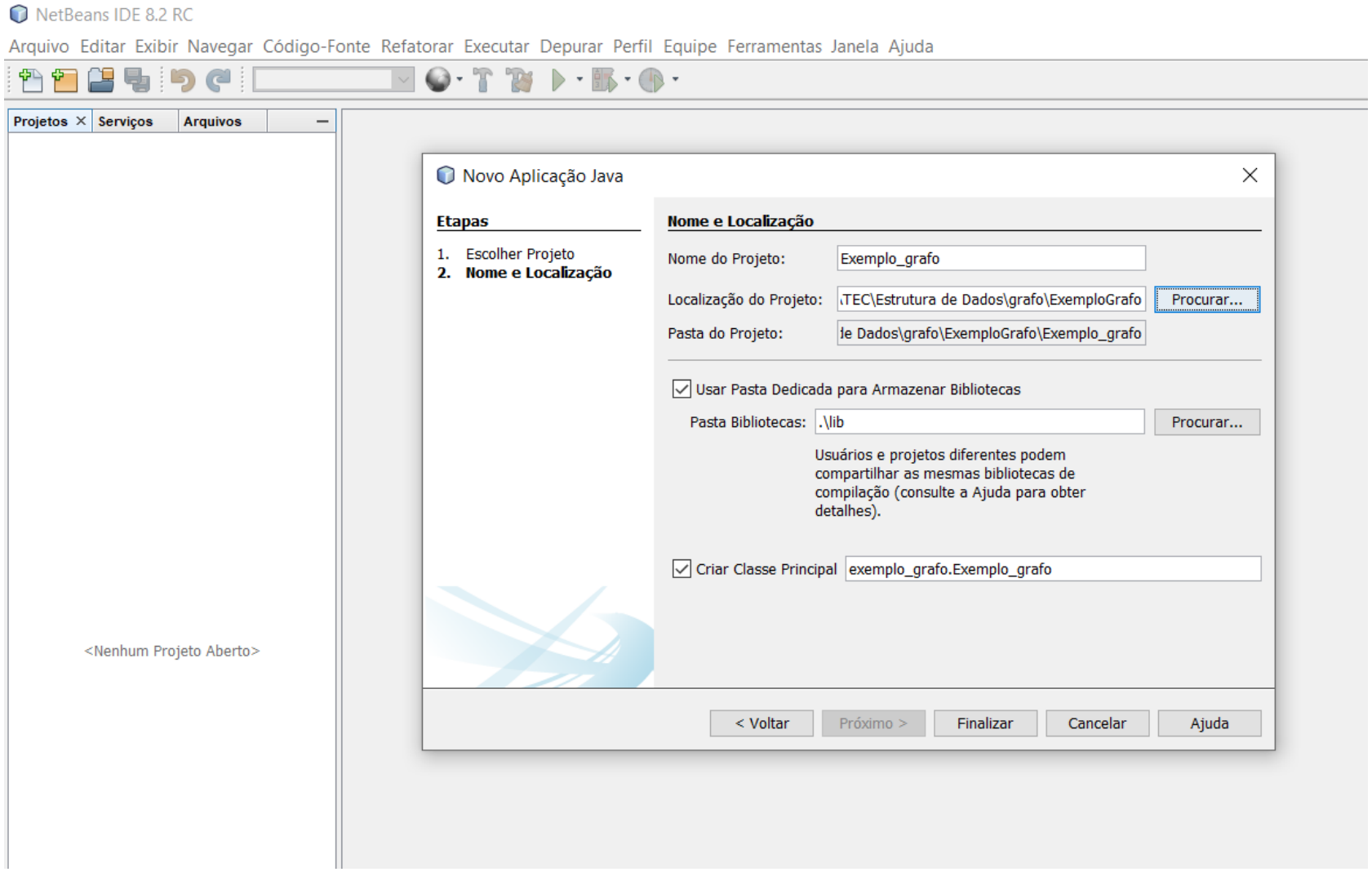
Grafos

1º Vamos criar um novo projeto



Grafos

1º Vamos criar um novo projeto



Grafos

2º Vamos criar nossas Classes para esse projeto:

- Arresta
- Vertice
- Grafo
- ExemploGrafo (Para testar a aplicação)

Grafos

1º Vamos criar um novo projeto

Grafos

Classe Aresta:

Aresta.java | Vertice.java | Grafo.java | ExemploGrafo.java

Código-Fonte | Histórico

```java
package exemplografo;

public class Aresta<TIPO> {
    private Double peso;
    private Vertice<TIPO> inicio;
    private Vertice<TIPO> fim;

    public Aresta(Double peso, Vertice<TIPO> inicio, Vertice<TIPO> fim){
        this.peso = peso;
        this.inicio = inicio;
        this.fim = fim;
    }
    public Double getPeso() {
        return peso;
    }
    public void setPeso(Double peso) {
        this.peso = peso;
    }
    public Vertice<TIPO> getInicio() {
        return inicio;
    }
    public void setInicio(Vertice<TIPO> inicio) {
        this.inicio = inicio;
    }
    public Vertice<TIPO> getFim() {
        return fim;
    }
    public void setFim(Vertice<TIPO> fim) {
        this.fim = fim;
    }
}
```

Grafos

Classe Vertice:

Aresta.java | Vertice.java | Grafo.java | ExemploGrafo.java

Código-Fonte | Histórico

```java
package exemplografo;

import java.util.ArrayList;

public class Vertice<TIPO> {
    private TIPO dado;
    private ArrayList<Aresta<TIPO>> arestasEntrada;
    private ArrayList<Aresta<TIPO>> arestasSaida;

    public Vertice(TIPO valor){
        this.dado = valor;
        this.arestasEntrada = new ArrayList<Aresta<TIPO>>();
        this.arestasSaida = new ArrayList<Aresta<TIPO>>();
    }

}
```

Grafos

Classe Vertice:

Métodos de acesso

Aresta.java | Vertice.java | Grafo.java | ExemploGrafo.java

Código-Fonte | Histórico

```java
    public TIPO getDado() {
        return dado;
    }

    public void setDado(TIPO dado) {
        this.dado = dado;
    }

    public ArrayList<Aresta<TIPO>> getArestasEntrada() {
        return arestasEntrada;
    }

    public ArrayList<Aresta<TIPO>> getArestasSaida() {
        return arestasSaida;
    }

    public void setArestasEntrada(ArrayList<Aresta<TIPO>> arestasEntrada) {
        this.arestasEntrada = arestasEntrada;
    }

    public void setArestasSaida(ArrayList<Aresta<TIPO>> arestasSaida) {
        this.arestasSaida = arestasSaida;
    }


    public void adicionarArestaEntrada(Aresta<TIPO> aresta){
        this.arestasEntrada.add(aresta);
    }

    public void adicionarArestaSaida(Aresta<TIPO> aresta){
        this.arestasSaida.add(aresta);
    }
}
```

Grafos

Classe Grafo:

Aresta.java | Vertice.java | Grafo.java | ExemploGrafo.java

Código-Fonte | Histórico

```java
package exemplografo;

import java.util.ArrayList;

public class Grafo<TIPO> {
    private ArrayList<Vertice<TIPO>> vertices;
    private ArrayList<Aresta<TIPO>> arestas;

    public Grafo(){
        this.vertices = new ArrayList<Vertice<TIPO>>();
        this.arestas = new ArrayList<Aresta<TIPO>>();
    }


}
```

Grafos

Classe Grafo:

Aresta.java | Vertice.java | Grafo.java | ExemploGrafo.java

Código-Fonte | Histórico

```java
1   package exemplografo;
2
3   import java.util.ArrayList;
4
5   public class Grafo<TIPO> {
        private ArrayList<Vertice<TIPO>> vertices;
        private ArrayList<Aresta<TIPO>> arestas;
8
9       public Grafo(){
            this.vertices = new ArrayList<Vertice<TIPO>>();
            this.arestas = new ArrayList<Aresta<TIPO>>();
12      }
13
14      public void adicionarVertice(TIPO dado){
            Vertice<TIPO> novoVertice = new Vertice<TIPO>(dado);
16          this.vertices.add(novoVertice);
17      }
18  }
```

Grafos

Classe Grafo:

Aresta.java | Vertice.java | Grafo.java | ExemploGrafo.java

Código-Fonte | Histórico

```java
    private ArrayList<Aresta<TIPO>> arestas;

    public Grafo(){
        this.vertices = new ArrayList<Vertice<TIPO>>();
        this.arestas = new ArrayList<Aresta<TIPO>>();
    }

    public void adicionarVertice(TIPO dado){
        Vertice<TIPO> novoVertice = new Vertice<TIPO>(dado);
        this.vertices.add(novoVertice);
    }

    public void adicionarAresta(Double peso, TIPO dadoInicio, TIPO dadoFim){
        Vertice<TIPO> inicio = this.getVertice(dadoInicio);
        Vertice<TIPO> fim = this.getVertice(dadoFim);
        Aresta<TIPO> aresta = new Aresta<TIPO>(peso, inicio, fim);
        inicio.adicionarArestaSaida(aresta);
        fim.adicionarArestaEntrada(aresta);
        this.arestas.add(aresta);
    }

    public Vertice<TIPO> getVertice(TIPO dado){
        Vertice<TIPO> vertice = null;
        for(int i=0; i < this.vertices.size(); i++){
            if (this.vertices.get(i).getDado().equals(dado)){
                vertice = this.vertices.get(i);
                break;
            }
        }
        return vertice;
    }
}
```

Grafos

Classe ExemploGrafo:

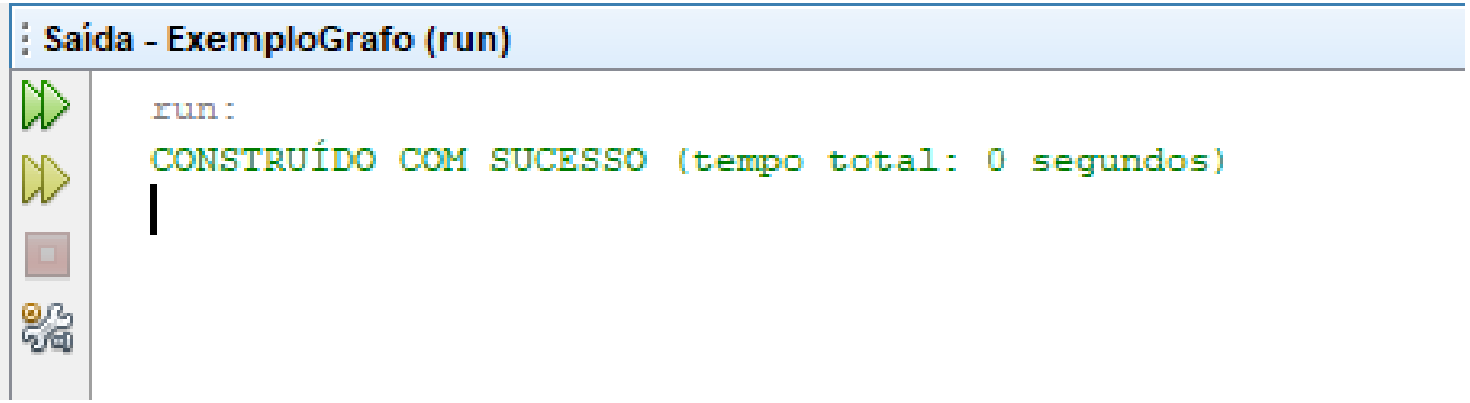
Aresta.java | Vertice.java | Grafo.java | ExemploGrafo.java

Código-Fonte | Histórico

```java
1   package exemplografo;
2
3   public class ExemploGrafo {
4
5
6       public static void main(String[] args) {
7           Grafo<String> grafo = new Grafo<String>();
8           grafo.adicionarVertice("Paulo");
9           grafo.adicionarVertice("Lorenzo");
10          grafo.adicionarVertice("Cleuza");
11          grafo.adicionarVertice("Kleber");
12          grafo.adicionarVertice("Claudio");
13
14          grafo.adicionarAresta(2.0, "Paulo", "Lorenzo");
15          grafo.adicionarAresta(3.0, "Lorenzo", "Kleber");
16          grafo.adicionarAresta(1.0, "Kleber", "Cleuza");
17          grafo.adicionarAresta(1.0, "Paulo", "Cleuza");
18          grafo.adicionarAresta(2.0, "Claudio", "Lorenzo");
19          grafo.adicionarAresta(3.0, "Claudio", "Paulo");
20
21      }
22  }
```

Grafos

Resultado:



Funcionou, porém não exibe os dados, para isso vamos fazer uma busca.

Grafos

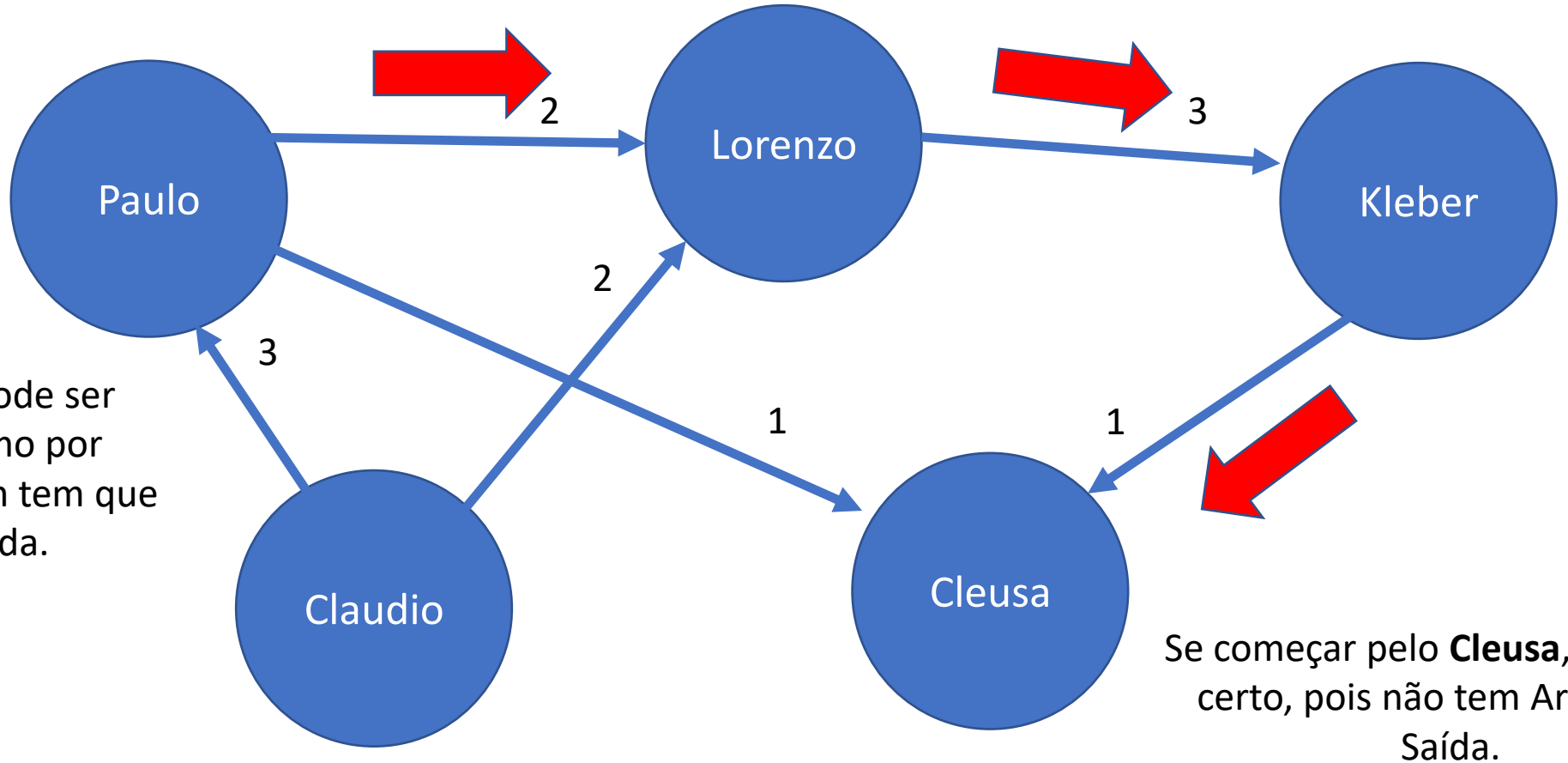
Quando trabalhamos com Grafo, podemos usar **Busca em Profundidade** ou **Busca em Largura**.

A **Busca em Profundidade** é um algoritmo utilizado para percorrer ou buscar itens dentro das estruturas de dados grafos ou árvores. Sua característica básica é percorrer todos os nós filhos ao nó raiz o mais profundo possível para somente depois retroceder.

Grafos

A Busca em Profundidade vai percorrer um elemento até onde é possível;

Exemplo:



O ponto de partida pode ser qualquer Vértice como por exemplo o **Paulo**, porém tem que ter Arestas de Saída.

Se começar pelo **Cleusa**, não daria certo, pois não tem Arestas de Saída.

Grafos

A **Busca em Largura** faz uma verificação sem a preocupação de se aprofundar (seguir as Arestas), ele faz uma verificação por todos os elementos válidos.

Formalmente, uma busca em largura é um método de busca não-informada (ou desinformada) que expande e examina sistematicamente todos os vértices de um grafo direcionado ou não-direcionado.

Grafos

Vamos criar uma **Busca em Largura** para o nosso projeto de Grafo em Java, para isso vamos criar um método na classe Grafo com o nome de **BuscaEmLargura()**.

Grafos

Vamos criar uma **Busca em Largura** para o nosso projeto de Grafo em Java, para isso vamos criar um método na classe Grafo com o nome de **BuscaEmLargura()**.

Grafos

Classe Grafo:

Aresta.java | Vertice.java | Grafo.java | ExemploGrafo.java

Código-Fonte | Histórico

```
38
39      public void BuscaEmLargura(){
            ArrayList<Vertice<TIPO>> marcados = new ArrayList<Vertice<TIPO>>();
            ArrayList<Vertice<TIPO>> fila = new ArrayList<Vertice<TIPO>>();
42          Vertice<TIPO> atual = this.vertices.get(0);
43          marcados.add(atual);
44          System.out.println(atual.getDado());
45          fila.add(atual);
46          while(fila.size() > 0){
47              Vertice<TIPO> visitado = fila.get(0);
48              for(int i=0; i < visitado.getArestasSaida().size();i++){
49                  Vertice<TIPO> proximo = visitado.getArestasSaida().get(i).getFim();
50                  if (!marcados.contains(proximo)){
51                      marcados.add(proximo);
52                      System.out.println(proximo.getDado());
53                      fila.add(proximo);
54                  }
55              }
56              fila.remove(0);
57          }
58      }
59  }
```

Grafos

Vamos Testar nossa busca:

Aresta.java | Vertice.java | Grafo.java | ExemploGrafo.java

Código-Fonte | Histórico

```java
package exemplografo;

public class ExemploGrafo {


    public static void main(String[] args) {
        Grafo<String> grafo = new Grafo<String>();
        grafo.adicionarVertice("Paulo");
        grafo.adicionarVertice("Lorenzo");
        grafo.adicionarVertice("Cleuza");
        grafo.adicionarVertice("Kleber");
        grafo.adicionarVertice("Claudio");

        grafo.adicionarAresta(2.0, "Paulo", "Lorenzo");
        grafo.adicionarAresta(3.0, "Lorenzo", "Kleber");
        grafo.adicionarAresta(1.0, "Kleber", "Cleuza");
        grafo.adicionarAresta(1.0, "Paulo", "Cleuza");
        grafo.adicionarAresta(2.0, "Claudio", "Lorenzo");
        grafo.adicionarAresta(3.0, "Claudio", "Paulo");

        grafo.BuscaEmLargura();

    }
}
```

Grafos

Resultado:

Saída - ExemploGrafo (run)

```
run:
Paulo
Lorenzo
Cleuza
Kleber
CONSTRUÍDO COM SUCESSO (tempo total: 0 segundos)
```

Grafos

Posso fazer uma alteração para determinar onde ele começa a nossa busca:

Claudio está na posição 4

Aresta.java | Vertice.java | Grafo.java | ExemploGrafo.java

Código-Fonte | Histórico

```java
1   package exemplografo;
2
3   public class ExemploGrafo {
4
5
6       public static void main(String[] args) {
7           Grafo<String> grafo = new Grafo<String>();
8           grafo.adicionarVertice("Paulo");
9           grafo.adicionarVertice("Lorenzo");
10          grafo.adicionarVertice("Cleuza");
11          grafo.adicionarVertice("Kleber");
12          grafo.adicionarVertice("Claudio");
13
14          grafo.adicionarAresta(2.0, "Paulo", "Lorenzo");
15          grafo.adicionarAresta(3.0, "Lorenzo", "Kleber");
16          grafo.adicionarAresta(1.0, "Kleber", "Cleuza");
17          grafo.adicionarAresta(1.0, "Paulo", "Cleuza");
18          grafo.adicionarAresta(2.0, "Claudio", "Lorenzo");
19          grafo.adicionarAresta(3.0, "Claudio", "Paulo");
20
21          grafo.BuscaEmLargura();
22
23      }
24  }
```

Grafos

Vou alterar a nossa busca para começar na posição 4:

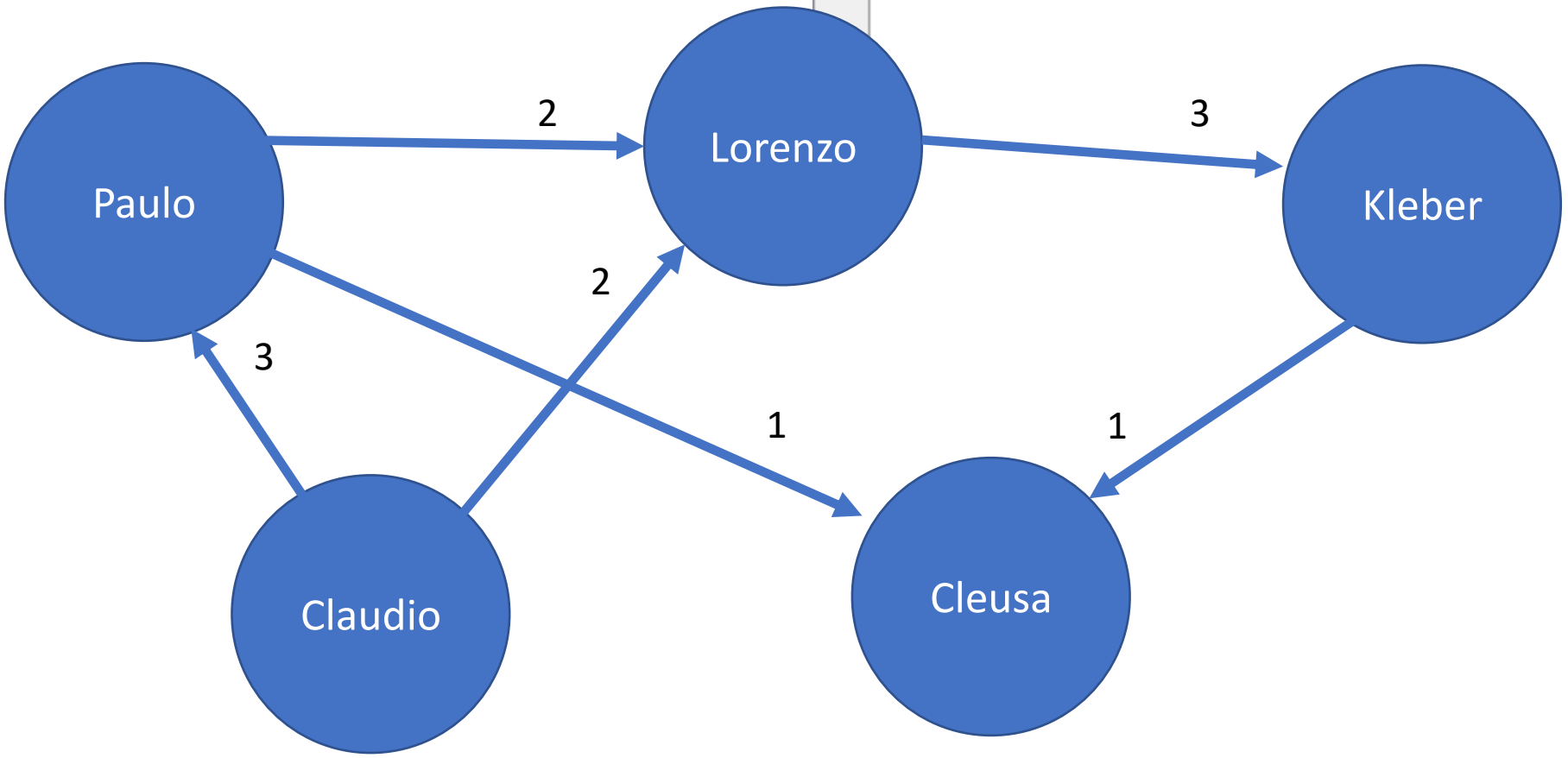
```
public void BuscaEmLargura(){
    ArrayList<Vertice<TIPO>> marcados = new ArrayList<Vertice<TIPO>>();
    ArrayList<Vertice<TIPO>> fila = new ArrayList<Vertice<TIPO>>();
    Vertice<TIPO> atual = this.vertices.get(4);
    marcados.add(atual);
    System.out.println(atual.getDado());
    fila.add(atual);
    while(fila.size() > 0){
        Vertice<TIPO> visitado = fila.get(0);
        for(int i=0; i < visitado.getArestasSaida().size();i++){
            Vertice<TIPO> proximo = visitado.getArestasSaida().get(i).getFim();
            if (!marcados.contains(proximo)){
                marcados.add(proximo);
                System.out.println(proximo.getDado());
                fila.add(proximo);
            }
        }
        fila.remove(0);
    }
}
```

Grafos

Vamos testar novamente:

Saída - ExemploGrafo (run)

```
run:
Claudio
Lorenzo
Paulo
Kleber
Cleuza
CONSTRUÍDO COM SUCESSO (tempo total: 0 segundos)
```



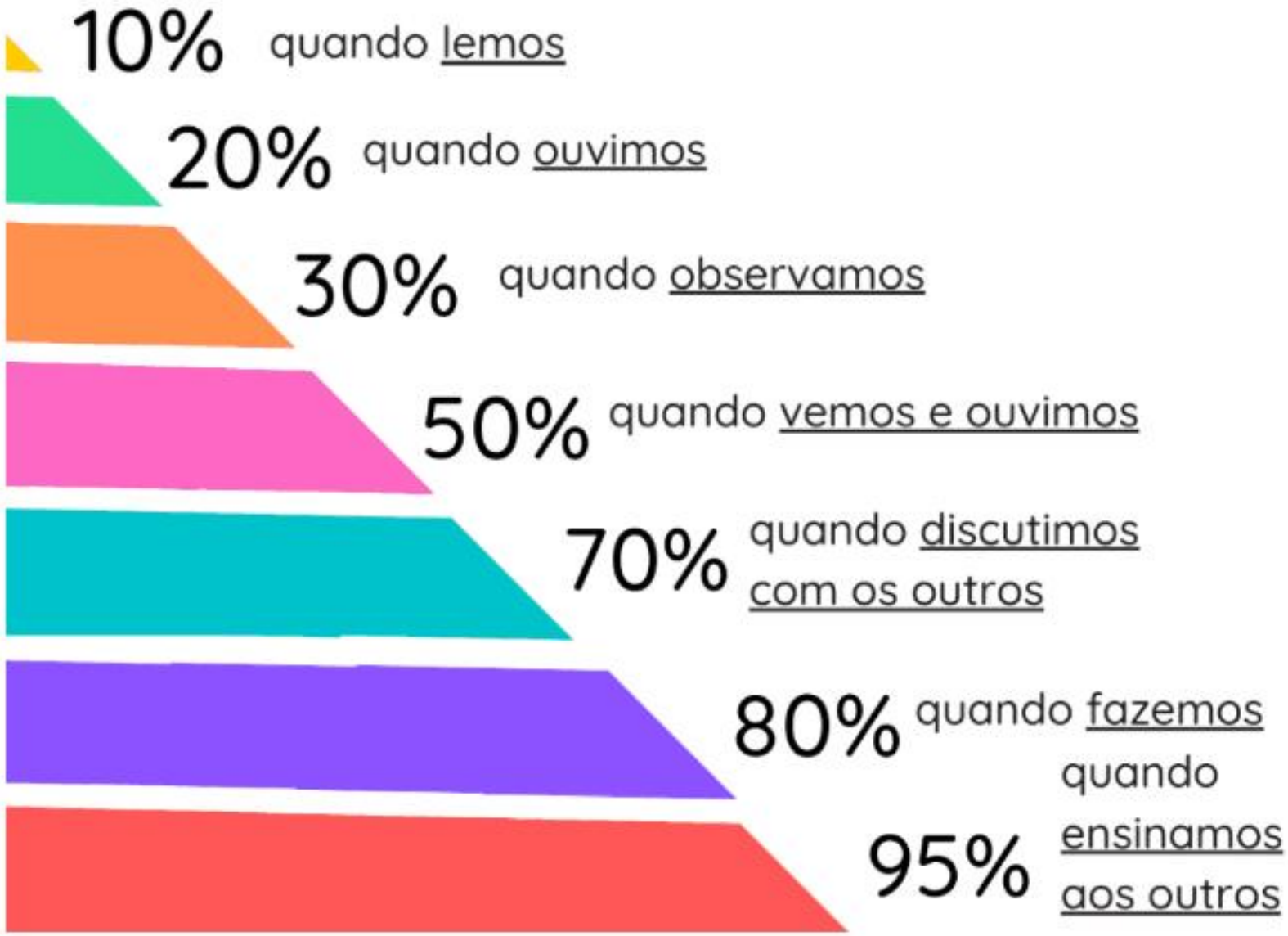
1.
Unidade

Grafos - Aula Invertida

Grafos – Aula Invertida

Se no **modelo tradicional** o professor em uma aula expositiva explica a matéria no quadro para que depois os alunos façam, sozinhos, a lição de casa, a **sala de aula invertida** (do inglês, *flipped classroom*) significa a inversão desta lógica: primeiro o aluno faz a internalização dos conceitos essenciais antes de aula e depois, junto à turma, discute os conhecimentos adquiridos e tira possíveis dúvidas de conteúdo com a ajuda e orientação do professor.

Grafos – Aula Invertida

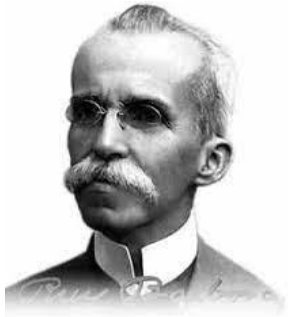


Grafos – Aula Invertida

Como uma Atividade ADO (Pode ser individual ou em grupo), você devem criar uma aplicação na linguagem Java utilizando a Estrutura de Dados Grafo.

- Utilizar o Power-Point para descrever os conceitos;
- Apresentar o Código Fonte e quando necessário utilizar comentários;
- Considerações finais sobre a aplicação.

"Maior que a tristeza de não haver vencido é a vergonha de não ter lutado"



Rui Barbosa

Obrigado!

Se precisar ...

Prof. Claudio Benossi

claudio.benossi@sp.senac.br